

Shopping Made Easy

Written Report by Karshin Sachdeva and Ethan Zhu from Haileybury College, Year 11

Note: Some images might be difficult to read so the code can be found on this [GitHub Page](#)

Aim:

Mobility is an ability adults and children take for granted since the ability to readily move over long distances becomes increasingly difficult with age. Senior citizens and those who have physical disabilities and require crutches have difficulties with their mobility. The aim of this invention is to allow individuals with such mobility difficulties to navigate around shopping malls or grocery stores in a timely manner.

Introduction:

Based on the Australian Institute of Health and Welfare, around 60% of people aged 85 years or above have a physical disability that potentially impairs their ability to move. Research has established that elderly citizens over the age of 80 can walk on average only 2 kilometres before feeling fatigued.

As a result, it is essential that they have access to technology that can help them efficiently navigate through shopping malls or grocery stores without feeling fatigued. Currently, navigation apps, such as Google Maps and Apple Maps, only provide the shortest path based on road networks. To navigate around shopping malls, there are only interactable turnstiles that provide the shortest path to a singular store. This is not an efficient form of navigation because it only finds the shortest path to a singular store and the path can only be accessed on a turnstile inside the mall itself. To solve this, the app allows users to input several stores and finds the shortest path between all the inputted stores. It also indicates the entrance the person should take to maximise efficiency. This allows those with mobility impairment to efficiently navigate around shopping malls.

Abstract:

The website, which can be utilised by seniors and by those with impaired mobility, incorporates meta-heuristic algorithms to find the shortest path around a shopping mall. The project integrates an accompaniment of the simulated annealing algorithm and a greedy heuristic A star (A*) algorithm, to find the optimal path around large shopping malls. The grocery stores or malls are mapped out using a graph abstract data type. Each individual store is presented as a singular node and the path between neighbouring stores represented by an edge. The weights of the edges are equal due to the extremely small distance between neighbouring stores. Once the user inputs all the stores they want to visit through a search bar, the simulated annealing algorithm and A* algorithm calculate the optimal path from a store's entrance to each of the stores. By including these algorithms, seniors and those with impaired mobility can efficiently navigate around shopping malls without exerting much effort.

This project was chosen because everyone should be able to navigate easily and efficiently in public spaces. In an ever-evolving and busy society, ease of mobility is a priority with the rise of more complex shopping malls popping up, spanning larger than before. Furthermore, it is essential that seniors and those with mobility impairment can efficiently navigate around shopping malls with comfort. Currently, most path finding apps, such as Google Maps, do not compute the shortest path between multiple target locations. Instead, they only provide the shortest path between a starting point and ending point. Access to this technology is currently limited so its implementation can be beneficial in various industries. Alongside ease of mobility, it can be integrated into apps such as Uber Eats which would allow drivers who are picking up groceries to maximise their efficiency by finding the most optimal path through a grocery store.

Operating Instructions:

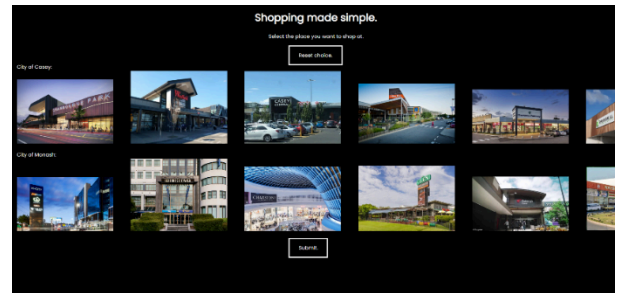
- 1) On a phone or laptop, click on the home.html page. This should open up a tab on a browser.
- 2) Once the splash screen has appeared, the main page with two sliders for different shopping malls will appear
- 3) By sliding on the sliders, press the shopping mall of choice and click the submit button
- 4) Once the submit button has been clicked, a page with a search bar will appear. The stores that want to be visited in that mall can be searched up and added to a list
- 5) Once all the stores that want to be visited are selected, click the button at the bottom of the screen
- 6) The algorithm will then calculate the shortest path based on the inputted stores and display the path on the map of the shopping mall
- 7) The map is interactable. If a new route with different stores want to be found, the back button can be clicked at the bottom of the page, which would direct the user back to the main page.

Design Brief:

Main Page:

In order to create the website, the programming languages HTML, CSS, and JavaScript were used. Whilst we first built the website using the online coding platform Codepen.io, we eventually moved the webpages to our local machines and set up individual HTML, CSS and JavaScript files, editing them using Visual Studio Code. This was done to allow for a more seamless user experience due to Codepen's feature of always keeping the text editors at the top of the screen. In addition, by hosting the webpage on our local machines, this allowed us to link the pages together when buttons were clicked to bring the user to different pages.

On the main page of the site, the layout is relatively simple with a title and some text below showcasing some instructions. The main features on this page are the image track that is able to be scrolled horizontally with by clicking and dragging and the ability to select and deselect malls that the person would like to shop at. When an image is clicked on, it becomes larger to signify that it has been selected, and only 1 mall at a time can be selected (i.e. when one mall image is clicked and is enlarged, clicking on another mall image will make the original enlarged one original size again and make the 2nd one). A reset button is also included to reset the current choice of mall, which will make any enlarged image regular size. After a user takes them to the 2nd page.



We were able to achieve the animation of enlarging and shrinking the images by utilising the following function that was ran on clicking of the image which enlarged 1 image and shrank the others (notice how only 1 image was scaled to 1.1 and the others were scaled to 1, this was done to prevent multiple images from being selected at once).

Note that we also add `draggable="false"` for each image. This is to prevent the image being able to be ‘dragged’ when clicked on.

```
src="img/signage.jpg" draggable="false" onclick="enlargeImg()" id="img1" />
```

For the reset button, all we had to do was reset the position of each image. This was a straightforward way to do as we just had to modify the above code a bit.

The home page, as stated previously, was made using HTML, CSS, and JavaScript. The title, texts and buttons were all written into the HTML, and so were the image tracks. We imported some fonts to make the text look more stylish from Google Fonts, and utilised line breaks with the `<br>` tag for formatting purposes. Below is a part of the code for the HTML from the home page.

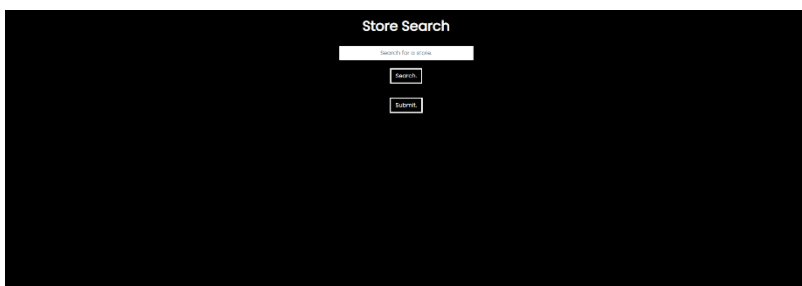
Utilising the HTML img tag, we were able to embed images within our webpage by entering the image source link. By placing all these images within a div which is within another div, we were able to create an “image track” which, along with the proper CSS, would allow people to scroll using their mouse on a formatted track.

To make the image track actually scroll, we utilise the events to determine the current state of the mouse. When we click

"mousedown", "mousemove", "mouseup" and "mouseleave" to determine the current state of the mouse. When the mouse is clicked, the current position of the mouse is recorded via the variables startX and scrollLeft. When the mouse moves, the distance between the start and current position is calculated via the walk variable and the image track is updated to be scrolled to this position. When the user lifts the click, the mouse returns to normal instead of being a grabbing icon and the scrolling is stopped.

Second Page:

After choosing their selected mall and clicking the submit button, users are taken to the 2nd page where they can select the shops that they would like to visit:



In order to create the search bar, the input HTML

tag was utilised. The list of stores that comes after up the user presses enter or the search button (see below) was made using a ul (unordered list) HTML tag. The buttons were made using the button HTML

```

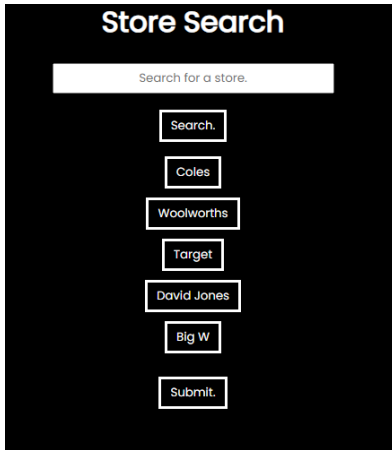
<script>
    function enlargeImg() {
        img = document.getElementById("img1");
        img.style.transform = "scale(1.1)";
        img.style.transition = "transform 0.25s ease";
        img2.style.transform = "scale(1)";
        img2.style.transition = "transform 0.25s ease";
        img3.style.transform = "scale(1)";
        img3.style.transition = "transform 0.25s ease";
        img4.style.transform = "scale(1)";
        img4.style.transition = "transform 0.25s ease";
        img5.style.transform = "scale(1)";
        img5.style.transition = "transform 0.25s ease";
        img6.style.transform = "scale(1)";
        img6.style.transition = "transform 0.25s ease";
        img7.style.transform = "scale(1)";
        img7.style.transition = "transform 0.25s ease";
        img8.style.transform = "scale(1)";
        img8.style.transition = "transform 0.25s ease";
        img9.style.transform = "scale(1)";
        img9.style.transition = "transform 0.25s ease";
        img10.style.transform = "scale(1)";
        img10.style.transition = "transform 0.25s ease";
        img11.style.transform = "scale(1)";
        img11.style.transition = "transform 0.25s ease";
        img12.style.transform = "scale(1)";
        img12.style.transition = "transform 0.25s ease";
    }

    function resetImg() {
        img.style.transform = "scale(1)";
        img.style.transition = "transform 0.25s ease";
        img2.style.transform = "scale(1)";
        img2.style.transition = "transform 0.25s ease";
        img3.style.transform = "scale(1)";
        img3.style.transition = "transform 0.25s ease";
        img4.style.transform = "scale(1)";
        img4.style.transition = "transform 0.25s ease";
        img5.style.transform = "scale(1)";
        img5.style.transition = "transform 0.25s ease";
        img6.style.transform = "scale(1)";
        img6.style.transition = "transform 0.25s ease";
        img7.style.transform = "scale(1)";
        img7.style.transition = "transform 0.25s ease";
        img8.style.transform = "scale(1)";
        img8.style.transition = "transform 0.25s ease";
        img9.style.transform = "scale(1)";
        img9.style.transition = "transform 0.25s ease";
        img10.style.transform = "scale(1)";
        img10.style.transition = "transform 0.25s ease";
        img11.style.transform = "scale(1)";
        img11.style.transition = "transform 0.25s ease";
        img12.style.transform = "scale(1)";
        img12.style.transition = "transform 0.25s ease";
    }
</script>

```

[illegible]

tag and were contained in separate divs for stylistic purposes.



In order to make the page display the stores searched, we used a constant variable and the code beside it:

```
const stores = [
  "Coles",
  "Woolworths",
  "Target",
  "David Jones",
  "Big W"
];
```

When the user clicks the search button or the enter button, the searchStores function is ran, which converts any input inside the search bar to a lower case as well as the stores inside the constant variable to ensure that no matter if the input was lower or upper case it would still detect it.

```
const searchInput = document.getElementById("searchInput");
const searchButton = document.getElementById("searchButton");
const storeList = document.getElementById("storeList");

searchButton.addEventListener("click", searchStores);

searchInput.addEventListener("keypress", function (event) {
  if (event.key === "Enter") {
    searchStores();
  }
});

function searchStores() {
  const searchTerm = searchInput.value.toLowerCase();
  const matchingStores = stores.filter(store =>
    store.toLowerCase().includes(searchTerm)
  );

  displayStores(matchingStores);
}

function displayStores(stores) {
  storeList.innerHTML = "";

  if (stores.length === 0) {
    const listItem = document.createElement("li");
    listItem.textContent = "No stores found.";
    storeList.appendChild(listItem);
  } else {
    stores.forEach(store => {
      const listItem = document.createElement("li");
      const button = document.createElement("button");
      button.textContent = store;
      button.addEventListener("click", () => {
        console.log(store);
      });
      listItem.appendChild(button);
      storeList.appendChild(listItem);
    });
  }
}
```

In order to display the store, the displayStores function is used. The displayStores function creates a list element and a button element for each of the stores found in searchStore. This allows for the buttons of the corresponding stores to be displayed.

The Shortest Path Algorithm:

The shortest path algorithm is composed of 1 class, 8 functions and 4 constant variables. Once the path is found, the shortest path is displayed on an interactive map.

To find the shortest path around a shopping mall, the shopping mall must be mapped out. This was done by using an adjacency matrix which needs to be hardcoded. On the right is an example for Fountain Gate Shopping Centre.

```
const storesGraph = {
  'switch_lifestyle': ['the_pancake_parlour', 'fountain_gate_hotel', 'kinn_thai_restaurant', 'fountain_gate_hotel_entrance'],
  'the_pancake_parlour': ['switch_lifestyle', 'the_sporting_globe', 'grillid'],
  'grillid': ['the_pancake_parlour', 'san_churro', 'nandos', 'cinema_elevator', 'dumplings_plus'],
  'cinema_elevator': ['grillid', 'nandos', 'schnitz', 'dumplings_plus'],
  'fountain_gate_hotel': ['kinn_thai_restaurant', 'switch_lifestyle', 'fountain_gate_hotel_entrance'],
  'kinn_thai_restaurant': ['fountain_gate_hotel', 'the_sporting_globe', 'the_pancake_parlour'],
  'the_sporting_globe': ['kinn_thai_restaurant', 'san_churro', 'the_pancake_parlour'],
  'san_churro': ['the_sporting_globe', 'nandos', 'grillid'],
  'nandos': ['san_churro', 'schnitz', 'cinema_elevator', 'grillid'],
  'schnitz': ['nandos', 'cinema_elevator', 'gami_chicken_and_beer'],
  'dumplings_plus': ['surf_dive_n_ski', 'gami_chicken_and_beer', 'grillid', 'cinema_elevator'],
  'surf_dive_n_ski': ['dumplings_plus', 'foot_locker', 'foot_locker_elevator'],
  'gami_chicken_and_beer': ['schnitz', 'gumany_gower'],
  'foot_locker_elevator': ['foot_locker', 'surf_dive_n_ski'],
  'foot_locker': ['foot_locker_elevator', 'surf_dive_n_ski'],
  'fountain_gate_hotel_entrance': ['fountain_gate_hotel', 'switch_lifestyle']
}
```

```
const storeLocations = {
  'switch_lifestyle': [-41.53688, -71.367188],
  'the_pancake_parlour': [-36.9059, -67.148438],
  'grillid': [-31.275755, -62.666016],
  'cinema_elevator': [-28.943518, -54.84375],
  'fountain_gate_hotel': [-48.953061, -59.765625],
  'kinn_thai_restaurant': [-44.619865, -55.195313],
  'the_sporting_globe': [-41.272971, -53.789063],
  'san_churro': [-37.186731, -53.4375],
  'nandos': [-33.455948, -51.328125],
  'schnitz': [-28.635155, -49.21875],
  'dumplings_plus': [-23.259514, -59.765625],
  'surf_dive_n_ski': [-18.327441, -57.65625],
  'gami_chicken_and_beer': [-23.885572, -47.460937],
  'foot_locker_elevator': [-5.600358, -73.300701],
  'foot_locker': [-10.64932, -54.492187],
  'fountain_gate_hotel_entrance': [-48.241524, -67.9342]
}
```

To properly map the path, each store is also assigned a coordinate location on the map.

Alongside this, there are instructions for store entrances, stores on the ground floor and elevators. Once these locations are all mapped out, the shortest path can be found.

Permutations Function:

The method to find the shortest path is dependent on the number of stores the user wants to visit. If the user wants to visit less than or equal to 6 stores, every single permutations of the stores' order is found and the A* algorithm is run to find which permutation provides the shortest path. The permutation algorithm is a recursive function that chooses a given store in the inputted list of stores that want to be visited and slices the rest of the stores to ensure every permutation is found. This means that if the user wants to visit 5 stores, 120 permutations will be obtained. For every permutation, the A* algorithm calculates the path the user must take and the overall distance.

```
function allPermutations(nodes){
  if (nodes.length == 0){
    return [];
  }
  else if (nodes.length == 1){
    return [nodes];
  }
  else {
    let possiblePaths = []
    for (let i = 0; i < nodes.length; i++){
      let permutationChange = nodes.slice(0,i).concat(nodes.slice(i+1));
      let all = allPermutations(permutationChange);
      for (let p in all){
        possiblePaths.push([nodes[i]].concat(all[p]));
      }
    }
    return possiblePaths;
  }
}
```

A* Algorithm:

A* is coded as a separate function. A* is a shortest path finding algorithm that utilises a heuristic to find the next store to visit. Using the permutations, the algorithm iterates through each of the stores in the permutation and finds the shortest path between each store. For example, if the user wanted to visit Kmart, Big W and Target, the algorithm would use the combination function to find all the permutations of the three stores. Then A* would find the path between each of the stores. If the permutation had the order of Big W first, Target second and Kmart third, the A* algorithm would then use the geolocation coordinates to find the path from Big W to Target. This may involve passing stores such as JD Sports, EB Games and Coles. The output of the A* algorithm would then be this path: Big W, JD Sports, EB Games, Coles and then Target.

A* Heuristic + Dijkstra's Algorithm:

To find this path, A* uses a heuristic to see which store to visit next. The heuristic in this algorithm is another algorithm called the Dijkstra algorithm. The Dijkstra algorithm also finds the shortest path between two nodes. However, Dijkstra is not always optimal but is a trustworthy heuristic because it is more likely to underestimate the distance than overestimate. Practically, A* runs Dijkstra's algorithm to find the distance from the current store to the final store. Using this and the distance from

```
function alter(storesGraph, start_node, end_node, first, state) {
  let open = [start_node];
  let close = [];
  let g = {};
  g[start_node] = 0;
  let parents = {};
  parents[start_node] = start_node;
  while (open.length > 0) {
    let current = null;
    for (let i = 0; i < open.length; i++) {
      let node = open[i];
      if (current == null || g[node] < g[current]) {
        current = node;
      }
    }
    if (current == null) {
      return null;
    }
    if (current == end_node) {
      return [current];
    }
    function dijkstra(storesGraph, start_node, end_node){
      let visited = [];
      let dist_to = {};
      let pq = new PriorityQueue();
      for (node in storesGraph){
        dist_to[node] = Infinity;
      }
      dist_to[start_node] = 0;
      for (neighbour in storesGraph[start_node]){
        pq.add(1, storesGraph[start_node][neighbour]);
      }
      while (pq.size() > 0){
        let element = pq.remove();
        let current_dist_to = element[0];
        let current_node = element[1];
        if (current_node in visited){
          continue;
        }
        dist_to[current_node] = current_dist_to;
        for (neighbour in storesGraph[current_node]){
          let next_dist_to = current_dist_to + storesGraph[current_node][neighbour];
          if (next_dist_to < dist_to[neighbour]){
            dist_to[neighbour] = next_dist_to;
            pq.add(1, neighbour);
          }
        }
        visited.push(current_node);
      }
      return [current_node];
    }
    open = open.filter(node => node !== current);
    close.push(current);
    let neighbours = storesGraph[current];
    for (let neighbour in neighbours){
      let next_dist_to = g[current] + storesGraph[current][neighbour];
      if (next_dist_to < g[neighbour]){
        g[neighbour] = next_dist_to;
        parents[neighbour] = current;
        open.push(neighbour);
      }
    }
  }
}
```

the first node, the store is given a score. This occurs for all neighbouring stores too. A* then visits the neighbouring store with the lowest score because a lower score indicates that it is more likely to be part of the shortest possible path.

[This link visually demonstrates A*](#). The blue line would be the shortest path and the red blocks are the nodes that are given a score by the heuristic.

Simulated Annealing:

If the user wishes to visit over 6 stores, a meta-heuristic approach is used to find the possible permutation of the path. Specifically, simulated annealing is used. Simulated annealing is composed of 3 functions and involves randomly producing a permutation, scoring the permutation based on its length and then altering the permutation so the optimal result can be found. The number of times simulated annealing is run is dependent on the 'temperature' and cooling temperature or 'alpha.' These values control how many times a new permutation is found.

```
function score(storeGraph, storeEntrances, state){
  let total = 0;
  let totalPath = [];
  let pq = new PriorityQueue();

  if (state.length == 0){
    for (entrance in storeEntrances){
      pq.add([star(storeGraph, storeEntrances[entrance], state[0], true, state[0]), star(storeGraph, storeEntrances[entrance], state[0], true, state[1])]);
    }
    return pq.pop();
  } else {
    for (entrance in storeEntrances){
      pq.add([star(storeGraph, storeEntrances[entrance], state[0], true, state[0]), star(storeGraph, storeEntrances[entrance], state[0], true, state[1])]);
    }
    total += pq.pop()[0];
    totalPath = [...totalPath, pq.pop()[1]];
    for (let i = 0; i < state.length; i++){
      total += star(storeGraph, state[i], state[i+1], false, state[0]);
      totalPath = [...totalPath, [...totalPath, state[i], state[i+1], false, state[1]]];
    }
  }
  return [total, totalPath];
}
```

To score each permutation, a scoring function is called. The function uses A* to find the overall path length of the permutation and the path the user would have to take for that specific permutation. The A* function is called in the scoring function.

```
function simulatedAnnealing(storeGraph, storeEntrances, initialState){
  let temperature = 1.0;
  let alpha = 0.99;
  let minTemp = 0.01;

  let currentState = initialState;
  console.log(currentState);
  let currentScore = score(storeGraph, storeEntrances, currentState[0]);
  let currentOverallPath = score(storeGraph, storeEntrances, currentState[1]);

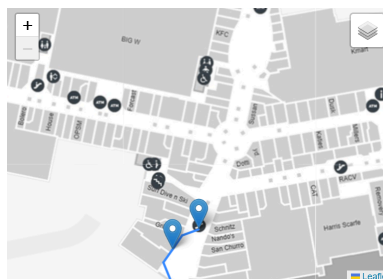
  while (temperature > minTemp){
    let newScore = score(storeGraph, storeEntrances, newScore[0]);
    let newOverallPath = score(storeGraph, storeEntrances, newScore[1]);

    if (newScore < currentScore){
      currentScore = newScore;
      currentState = newScore[0];
      currentOverallPath = newOverallPath;
    } else {
      let acceptanceProbability = Math.exp((newScore - currentScore) / temperature);
      if (Math.random() < acceptanceProbability){
        currentScore = newScore;
        currentState = newScore[0];
        currentOverallPath = newOverallPath;
      }
    }
    temperature = temperature * alpha;
  }
  return currentOverallPath;
}
```

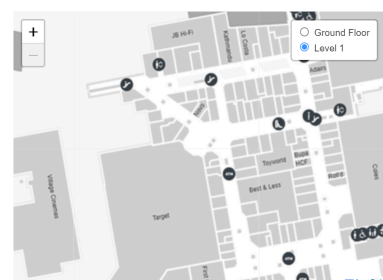
worse, it is run through an acceptance probability function, which will still accept the solution if it passes a random check. This probably of it being accepted is determined by the score: the better, the more likely it is to be accepted. This allows for a more global search and to avoid getting stuck in a local maxima.

Once the simulated annealing function stops annealing, the path based on the final permutation with the lowest score is outputted.

After the path has been discovered, it is displayed on an interactive map. The map is created with the 'Leaflet' module. An example of the map outputted is below for Fountain Gate.



In the example above, the user wants to visit Pancake Parlour and then go to the cinemas, so the algorithm recommends the user to enter through the entrance closest to Fountain Gate Hotel to then visit Pancake Parlour before going on the escalator to the cinemas. If a store has greater than one floor, the path will lead the user to an escalator to travel to the next floor. Once the user reaches the escalator, they can press the marker which changes the map to the floor they are travelling onto. The image below is the second floor of Fountain Gate. This map will appear once the marker for the escalator is clicked. This ensures that paths can be found for multi-story shopping malls.



The example above is a simple example which helps understand the program. The algorithm can find long paths that span shopping malls and requires a floor change multiple times.

Discussion & Conclusion:

Computational theory is heavily displayed in the project through the algorithms chosen. Simulated annealing, a meta-heuristic approach to finding the order of stores at which the person should visit, is accompanied with the A-star algorithm to tackle the Travelling Salesman Problem. This problem involves finding the optimal path between a set of different nodes in a graph, which is accomplished by the app. The major issue with the Travelling Salesman Problem is that computers struggle to find the optimal result in a tractable time. To overcome this hurdle, specifications to the named algorithms were made. The iterations that simulated annealing performs is precisely controlled by the temperature and cooling factor assigned. Since the A-star algorithm is part of the simulated annealing scoring function, it is responsible for most of the processing time. To ensure users are not waiting for too long and that the app holds their attention due to A star if they select a large number of stores, the iterations performed by simulated annealing are controlled. However, to ensure the app is considered computationally 'tractable' under Cobham's Thesis, optimality is sacrificed as the number of stores that need to be visited increases. This means that optimal results are found when the user wants to find the shortest path between 8 stores, but this optimality is lost once the number increases to 30 or 40 (although still made find a good-enough solution!). In practicality, it is unlikely that users would ever want to specifically find the shortest path between 40 stores, so this optimality has no major real-world impact.

The simulated annealing algorithm is only run if the user wants to visit 7 or more specific stores. This is purposeful. Time-complexity is a computational constraint on any app so managing this burden is a significant task. Based on numerous test trials, it was discovered that the optimal shortest path between 6 stores could be found in a short enough time by generating all possible combinations and finding the best option. When this number increases to 7, 7! (5040) different combinations are created. Modern computers are unable to run the A star algorithm on 5040 different combinations in a short time span convenient to the user. To tackle this issue, the simulated annealing algorithm is run instead which purposefully performs just over 6! iterations. This is a form of computational underestimation, but it does not have any impact on user experience in general. It ultimately outputs the shortest path quickly and efficiently.

One problem that occurred when coding the invention was difficulty modelling stores to apply the simulated annealing and A star algorithms. Originally, it was hoped that the store of the map can be overlayed with nodes and edges. However, this was difficult to achieve because JavaScript does not have an easily accessible graph abstract data type so a class would have to be coded. To achieve this, a node class would have to be created which would require manual efforts that take an overly long period of time. Instead, an adjacency matrix is used. The adjacency matrix includes every single store, elevator or bathroom in a shopping mall as a key and the neighbouring stores as a value. This avoids the need to map stores with nodes and edges.

When the solution was ideated, research was performed online and in-person. Scientific studies had been performed in Australia revolving around senior citizens in shopping malls. These studies provided information on their reasoning behind visiting shopping malls and the length of their stay. It helped prove that the app could be beneficial when applied in the real-world. Simultaneously, the idea was proposed to relatives with slight mobility impairment and positive feedback was provided. An understanding was formed that elderly normally require to sit in the chairs in between stores when at shopping malls and that buying several items at once was a burden. This helped direct the solution and how it was coded since it was a priority to ensure these issues were tackled.

Currently, the biggest limitation of the solution is that it is only accessible via a webpage. To solve this, the website will be converted into an app that can be accessed on Android and iOS devices by utilising platforms such as Apache Cordova. Another option is to use Android Studio and Apple's Developer apps to individually create the apps on each platform instead of using the generalised form in Apache Cordova. By having an app, users will be able to more easily access the services provided. This means there is a better user experience and that the solution is more accessible. Along with this, the structure of the website will have to be manipulated to fit the format of an app instead of a website. This would require buttons to be removed and replaced with singular clicks to open successive pages. By implementing these strategies, the solution becomes more accessible and easier to use for elderly and those with mobility impairment. It especially makes it easier for senior citizens who do not have a great understanding of technology. Overall, this project successfully helps those with mobility difficulties to navigate large spaces such as shopping malls.

References:

- <https://www.aihw.gov.au/reports/disability/people-with-disability-in-australia/contents/people-with-disability/prevalence-of-disability>
- https://www.researchgate.net/publication/274475594_Seniors_in_shopping_centres

CODE CAN BE FOUND ON THIS **GITHUB** PAGE: <https://github.com/karsh-s/ShoppingMadeEasy>